LibSpace – Library Management Backend System

1. Team Introduction

Team ID:SWTID-2026-9008

Team leader:

Name: Ahalya C H

Email : chundikottaih@gmail.com

Team members:

Name: R Aarthy

Email: aarthyaarthy1616@gmail.com

Name: S Dharshini

Email:ttamil0842@gmail.com

Name : S Hemavathy

Email:hemavathi9306@gmail.com

2. Project Introduction

LibSpace is a backend library management system developed using Node.js and Express.js. It allows users to register, login and manage books using REST APIs. The project demonstrates authentication using JWT and database operations using MongoDB.

3. Project Objectives

1 Build a secure user authentication system

2 Implement JWT based authorization

3 Develop CRUD operations for books

4 Maintain relationship between users and books

5 Provide REST API endpoints for library operations

4. Technologies Used

Technology	Purpose
Node.js	Backend runtime
Express.js	API framework
MongoDB	Database
Mongoose	MongoDB ODM

JWT	Authentication token
bcrypt	Password hashing
Postman	API testing

5. System Architecture

The application follows a layered architecture consisting of Routes, Controllers and Models. Routes handle incoming API requests, controllers contain business logic and models interact with the MongoDB database.

6. Folder Structure

1 controllers – Contains business logic

2 models – Database schemas

3 routes – API routes

4 middleware – Authentication middleware

5 config – Database configuration

6 server.js – Main server file

7. Setup Instructions

1 Install Node.js and MongoDB

2 Open the project folder in VS Code

3 Run npm install to install dependencies

4 Create a .env file with PORT and JWT_SECRET 5 Run npm start to start the server

8. API Endpoints

1 POST /api/auth/register – Register new user

2 POST /api/auth/login – Login user

3 POST /api/auth/logout – Logout user

4 GET /api/books – Get all books

5 GET /api/books/:id – Get book by ID

6 POST /api/books – Add book

7 PUT /api/books/:id – Update book

8 DELETE /api/books/:id – Delete book

9. Authentication Flow

Users first register and create an account. After logging in successfully, the server generates a JWT token. This token must be included in the Authorization header when accessing protected routes.

10. Testing

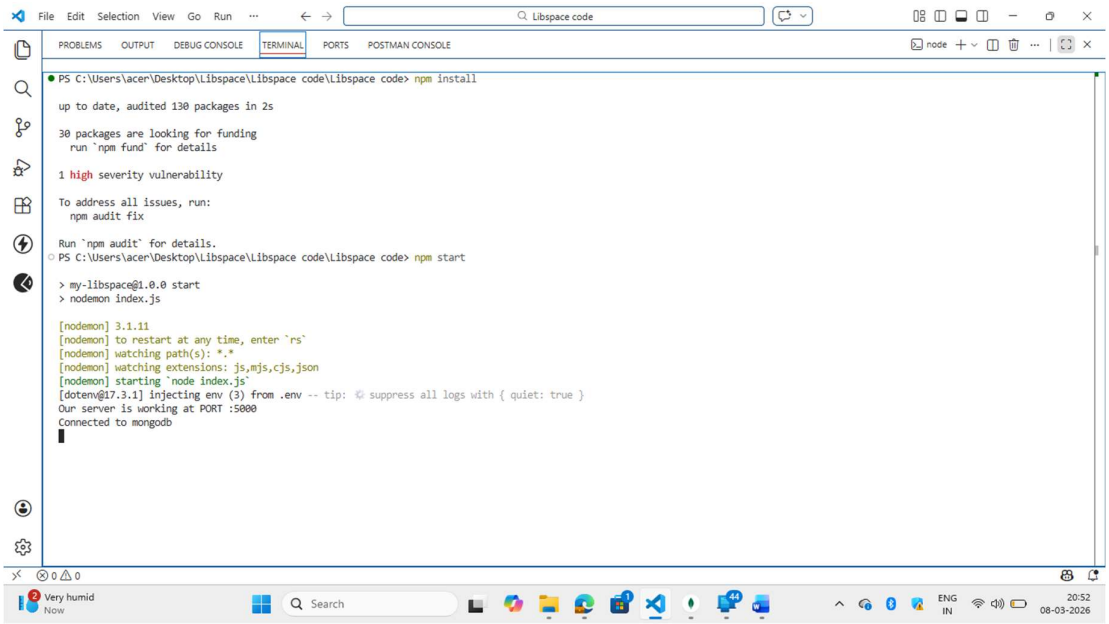
API endpoints are tested using Postman. Each endpoint is verified to ensure that authentication, database operations and responses work correctly.

11. Conclusion

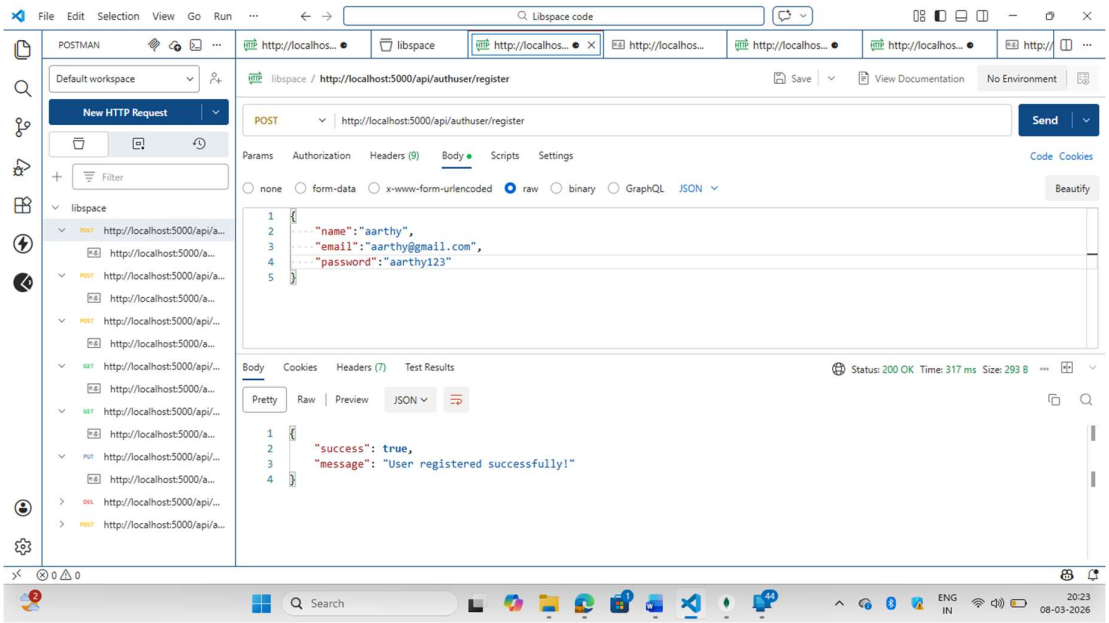
LibSpace demonstrates how backend systems handle authentication, API design and database operations. The project provides a simple but effective example of building a secure REST API using Node.js and MongoDB.

11.Screenshots:

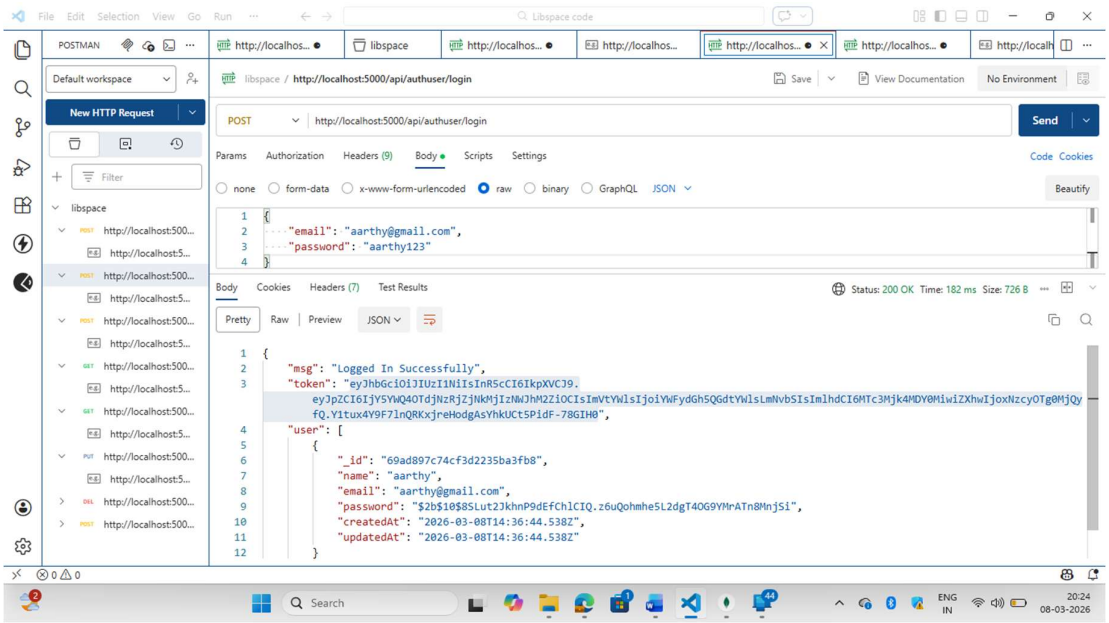
Output on terminal once server starts:



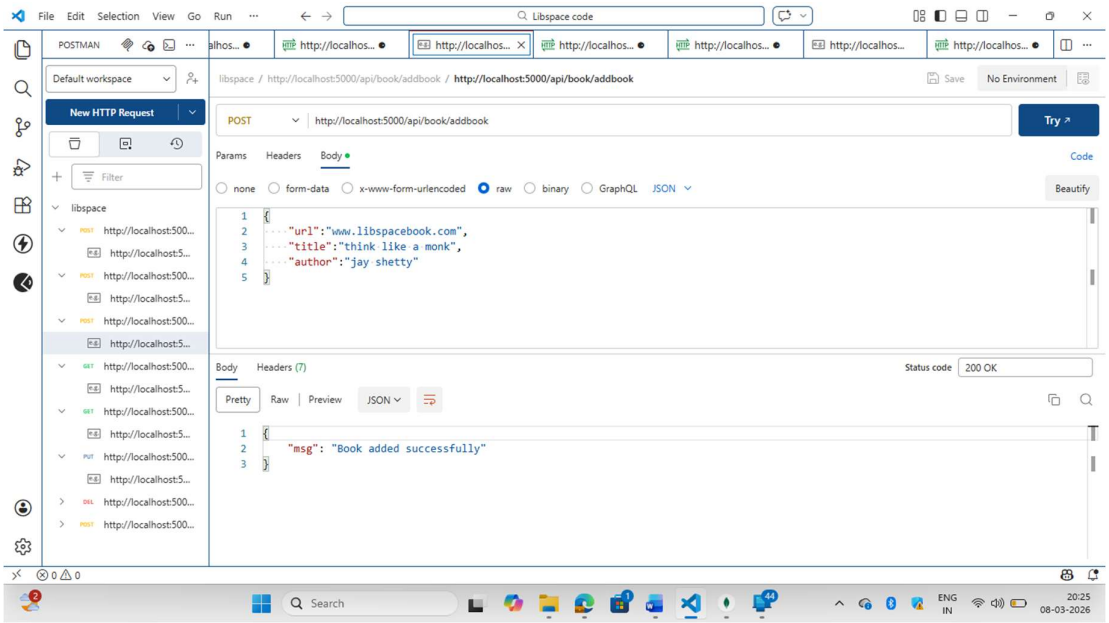
User registration output:



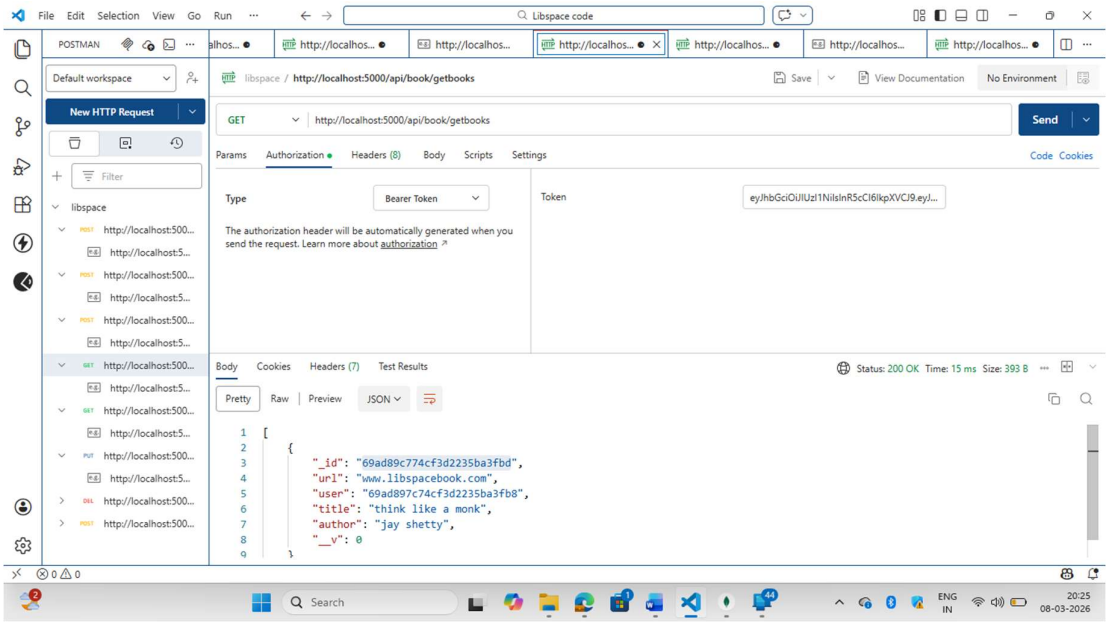
User Login output:



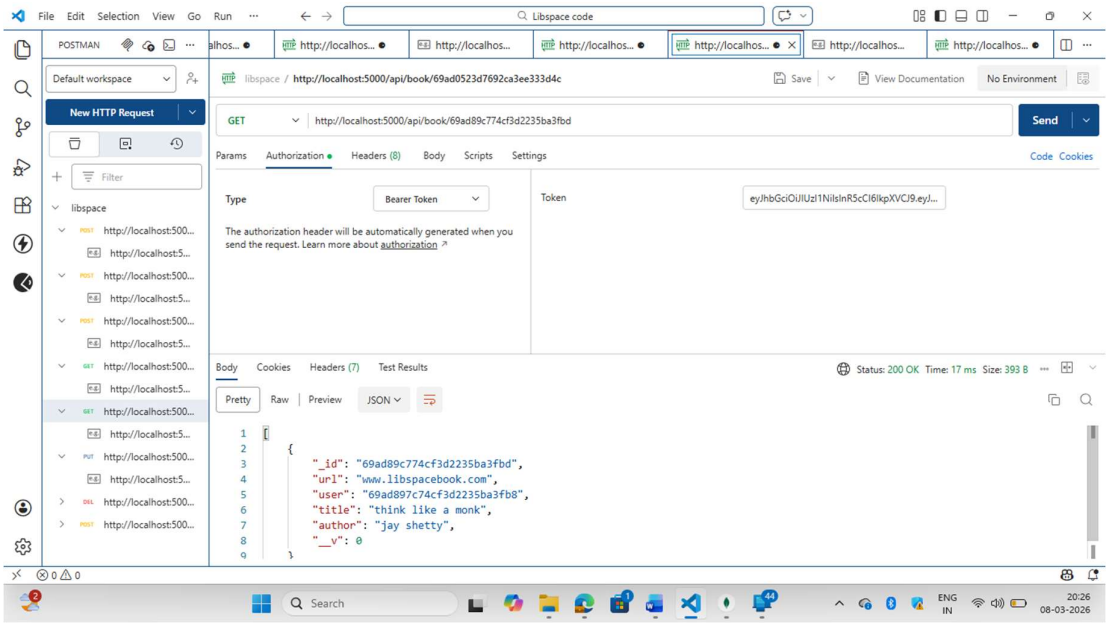
Adding book:



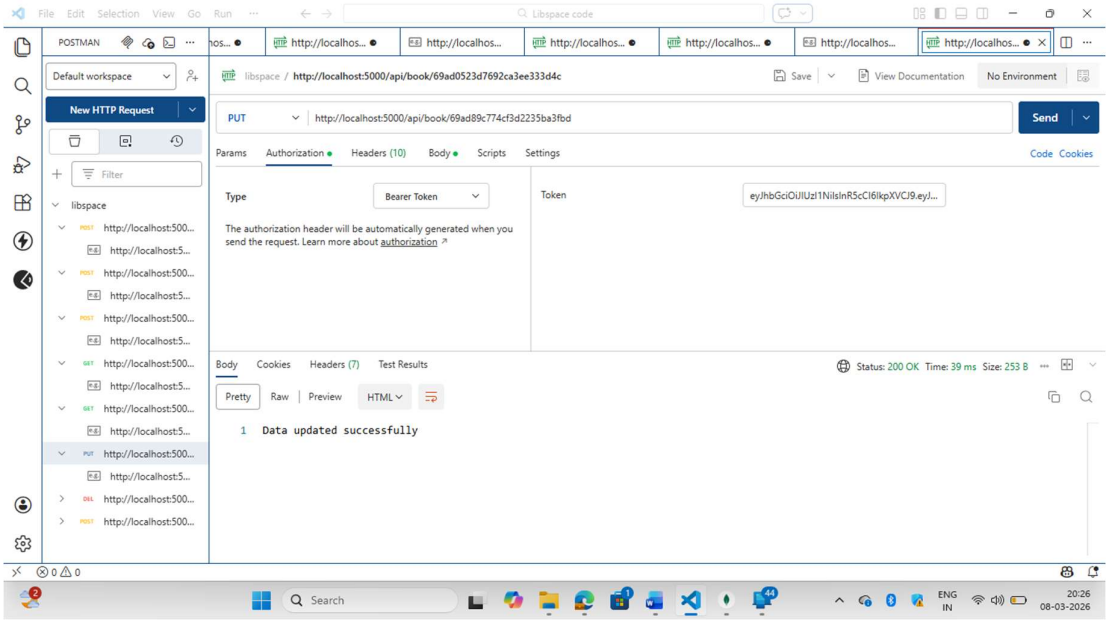
Get All books:



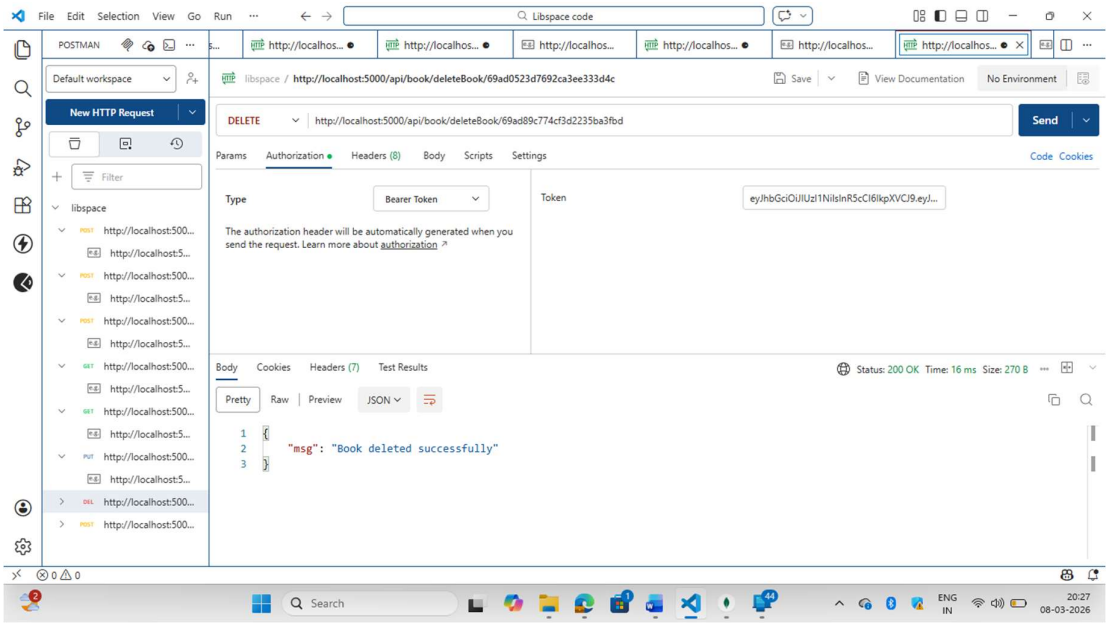
Get book by ID:



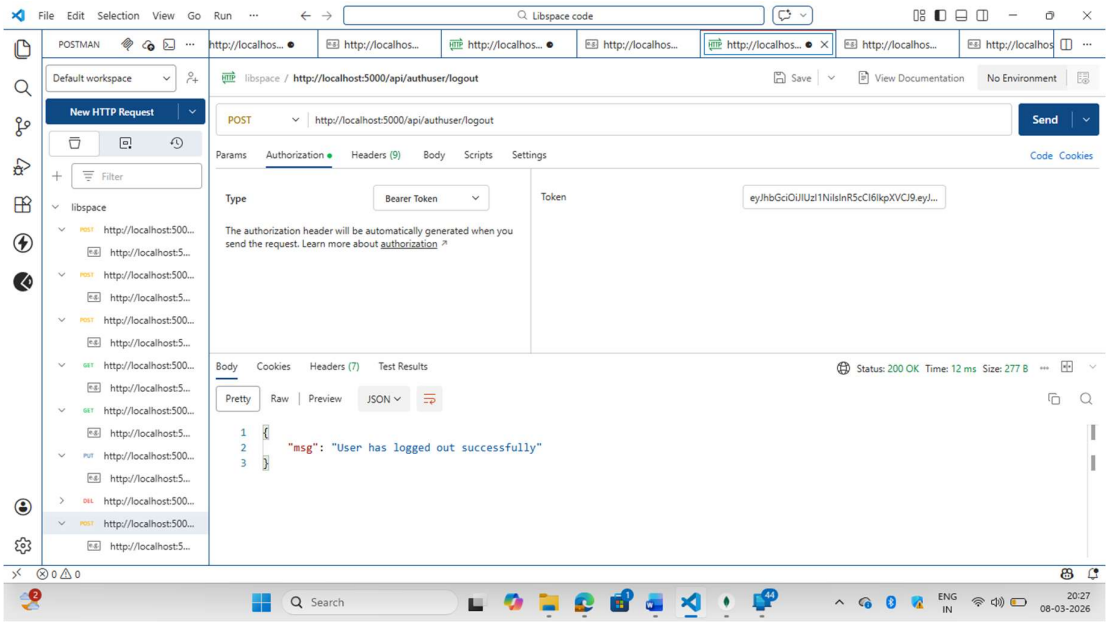
Update book:



Delete book:


File Edit Selection View Go Run
Libspace code
POSTMAN
Default workspace
New HTTP Request
libspace / http://localhost:5000/api/book/deleteBook/69ad0523d7692ca3ee333d4c
Save
View Documentation
No Environment
DELETE
http://localhost:5000/api/book/deleteBook/69ad89c774cf3d2235ba3fbd
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Code
Cookies
Type
Bearer Token
Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...
The authorization header will be automatically generated when you send the request. Learn more about authorization
Body
Cookies
Headers (7)
Test Results
Status: 200 OK Time: 16 ms Size: 270 B
Pretty
Raw
Preview
JSON
{
"msg": "Book deleted successfully"
}
Search
ENG IN
20:27
08-03-2026

User Logout:


File Edit Selection View Go Run
Libspace code
POSTMAN
Default workspace
New HTTP Request
libspace / http://localhost:5000/api/authuser/logout
Save
View Documentation
No Environment
POST
http://localhost:5000/api/authuser/logout
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
Type
Bearer Token
Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...
The authorization header will be automatically generated when you send the request. Learn more about authorization
Body
Cookies
Headers (7)
Test Results
Status: 200 OK Time: 12 ms Size: 277 B
Pretty
Raw
Preview
JSON
{
"msg": "User has logged out successfully"
}
Search
ENG IN
20:27
08-03-2026